## Assignment Number - 4 : Lane Departure System in Self Driving Cars

*Course Title: Computer Vision*
*Course Instructor: Dr. Mohsen Ali*

---

### Guidelines:

- Submit all of your code and results in a single zip file with name FirstName_RollNumber_04.zip
- Submit single zip file containing
  a) code b) results c) readme d) report
- There should be Readme.txt explaining how to run your code
- There should be a Report.pdf detailing your experience and highlighting any interesting result.
- Email instructor or TA if there are any questions. You cannot look at others code or use others code, however you can discuss with each other.
- Deadline is Sunday 4th January 2022 before 11:59 pm.

### Overview:

In this assignment we are going to learn about the implementation of a very important component of self-driving car called lane detection. We rely on accurate observation of various markings to help us drive, many of which have been standardized. One such marking is the lane-lines, detecting them allows us to keep our vehicle inside the lane. And identifying their type help us know how to share the road with other vehicles, e.g. changing lanes safely[1]. Detecting them automatically is a challenging task that must be solved by the self-driving or driver assisting system.

# Implementation

There are some basic steps for the implementation of lane detection that are discussed below. But you are not restricted to follow the same steps, we encourage you to put your own thoughts and come up with a better algorithm to solve this problem. *The algorithm steps have been adopted from the Udacity Self-driving car – Project 1*.

1. Convert the input RGB image (I) to the HSV space. Create a binary image B of the same size (row and column) of RGB image. Set all the pixels in the B to 1.

   HSV, it is basically a different way to describe color than the classic RGB we are used to, you can learn more about it on Wikipedia. We will explain later why this conversion is necessary.

[1] https://www.ctpgujranwala.pk/road-marking/



RGB HSV

Figure 1: Images in RGB and HSV color space

2. Remove the noise by applying a Gaussian filter. This should also remove small artifacts and small particles.
3. Filter out (turn black, set to zero) any pixel (in the binary image) that is not the color of the lanes (in HSV space), in our case white or yellow. Because lines on the road are marked with yellow or white color. You should get a result as in Fig. 2.

You will notice that HSV has allowed us to remove unwanted pixels with much more ease. We have to look at both Hue and Saturation of the pixel (which color it is and how intense the color is), giving us a room of adjustment for "value" (how dark it is). This setup allows us to handle shadows and lighting variations. To extract the yellow and white color you will need to study HSV color space. Also have a look at this table.



Figure 2: Output of step 3, filter out the pixel that are not color of lanes

There only remain ones with white or yellowish tint, with some false positives but our two lines are very clearly defined.

4. Convert the input image I to the gray scale image Ig. Use the Canny edge detector to detect the edges. The Canny algorithm detects edges on a picture by looking for quick changes in color between a pixel and its neighbors, in this case between the white or yellow lane line and black pixels. Use the binary image B to remove all the edge pixels (set to zero) which are set to zero in I.



Figure 3: Output of Canny Edge Detection

5. Now we define the region of interest. Given the position and orientation of the camera, you know that the lanes will be located in the lower half of the image, usually in a trapezoid covering the bottom corners and the center. You don't want your region to be too narrow and have the lines out of our region of interest.
   Assumption here is that the camera remains in constant place and lanes are flat, so that we can "guess" the region of interest. Mention this in your report.

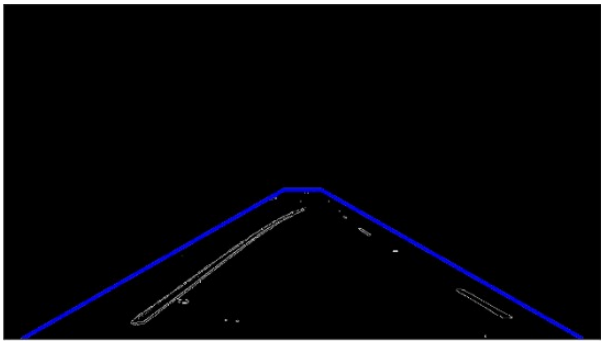


Figure 4: Region of Interest

6. Run Hough transform to detect lines on our image. What does Hough transform do? To summarize quickly, it allows us to easily extract all the lines passing through each of our edge points and group by similarity, thus grouping together the points that are roughly located on a same line.

   After the above step we will still get multiple lines, but we only want 2 distinct lines (to form the lane) for our car to drive in between.

7. Filter the lines that only have horizontal slopes. Based on the value of slopes, distinguish and group the lines into left and right lines.

   **Left Lane:** As the values of column coordinate increases , the values of rows coordinate decreases. So the gradient must be negative.

   **Right Lane:** As the values of column coordinate increases , the values of rows coordinate increases. So the gradient must be positive.

8. Last step is to compute the linear regression of both groups and draw the results on your region of interest. Simply, linear regression is an attempt at finding the relationship between a group of points, so basically finding the line that passes at the closest possible distance from each point. This operation will allow you to fill the blanks in a dashed lane line.



(a) Output of line fitting (b) Final output

Figure 5: Output of step 8

## Tasks:

For Assignment 4, you have to implement lane line detection. Test Images are given in the assignment folder. Create separate functions for all the steps.

For each image you have to submit:

- Images showing the output of step 3 given above
- Images showing your region of interest (output of step 5)
- Images showing the final output.
- Think about how to choose ROI automatically to include the true line only.